

Tyr-Crypto

Computational Handbook

Volume 3: SHA3 and SHAKE

A guided calculation book for the Keccak 5 by 5 state, sponge absorption, domain padding, theta, rho, pi, chi, iota, 24 rounds, and extendable output.

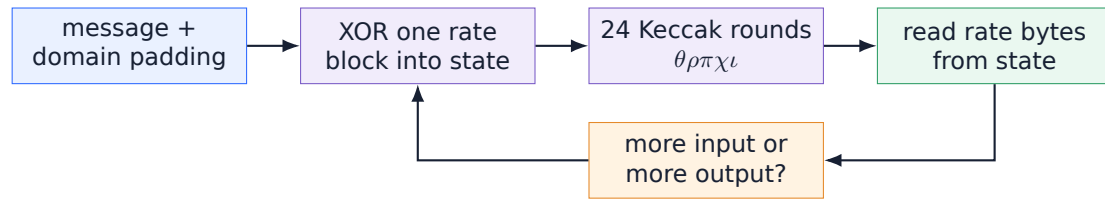
Real SHA3 uses 64-bit lanes. Small 8-bit lanes are used for hand calculations while preserving the exact movement and Boolean rules.

Blue	new idea, input, or plain-language explanation
Purple	exact recipe to follow
Green	fully worked calculation
Orange	exercise for the reader
Red	misuse, security, or implementation danger
Teal	checkpoint before continuing

Contents

1	The entire algorithm before the details	2
2	Method 1: locate a lane in the 5 by 5 state	3
3	Method 2: separate rate and capacity	4
4	Method 3: absorb one block	5
5	Method 4: add domain suffix and final padding bit	6
6	Method 5: apply theta - mix information between columns	7
7	Method 6: apply rho - rotate every lane by its assigned distance	8
8	Method 7: apply pi - move lanes to new coordinates	9
9	Method 8: apply chi - nonlinear row mixing	10
10	Method 9: apply iota - inject the round constant	11
11	Method 10: run one complete Keccak round	12
12	Method 11: squeeze output bytes	13
13	Method 12: choose SHA3 or SHAKE and follow the whole sponge	14
14	Reading the Tyr-Crypto implementation	15
15	Cumulative exercises	16
16	Solutions	17
17	References and code map	18

1 The entire algorithm before the details



Plain-language explanation

A sponge has a fixed-size internal state. It absorbs input into only the public-facing part called the **rate**. The hidden remainder is called the **capacity**. After every full input block, a permutation mixes all state bits. Output is later squeezed from the rate.

Do not skip this warning

SHA3 and SHAKE use the same Keccak-f[1600] permutation but different domain suffixes and output rules. Ethereum's historical Keccak-256 is not byte-for-byte the same function as standardized SHA3-256.

2 Method 1: locate a lane in the 5 by 5 state

Input: coordinates (x, y)
Do: compute $x + 5y$
Output: one array index

Plain-language explanation

The real state has 25 lanes of 64 bits: $25 \cdot 64 = 1600$ bits. Tyr-Crypto stores them in a flat array. Coordinate x moves across a row; y moves down rows.

Mechanical rule

$$\text{index}(x, y) = x + 5y, \quad 0 \leq x, y < 5.$$

Worked example 1

$$(0, 0) \rightarrow 0 + 5 \cdot 0 = \boxed{0}.$$

Worked example 2

$$(4, 0) \rightarrow 4 + 0 = \boxed{4}, \text{ the last lane of the first row.}$$

Worked example 3

$$(0, 3) \rightarrow 0 + 15 = \boxed{15}, \text{ the first lane of the fourth row.}$$

Worked example 4

$$(2, 4) \rightarrow 2 + 20 = \boxed{22}.$$

Exercises

Find indices for (a) $(1, 1)$, (b) $(3, 2)$, (c) $(4, 4)$, (d) recover coordinates from index 17.

3 Method 2: separate rate and capacity

Input: chosen SHA3 or SHAKE variant
Do: use its fixed rate; capacity is $1600 - r$
Output: input block size and hidden state size

Mechanical rule

$r + c = 1600$ bits. Rate bytes equal $r/8$. A larger capacity leaves a smaller rate and usually a larger security margin.

Worked example 1 - SHA3-224

Rate 144 bytes = 1152 bits; capacity $1600 - 1152 = \boxed{448}$ bits.

Worked example 2 - SHA3-256

Rate 136 bytes = 1088 bits; capacity = $\boxed{512}$ bits.

Worked example 3 - SHA3-384

Rate 104 bytes = 832 bits; capacity = $\boxed{768}$ bits.

Worked example 4 - SHA3-512

Rate 72 bytes = 576 bits; capacity = $\boxed{1024}$ bits.

Checkpoint

Tyr-Crypto uses 168 rate bytes for SHAKE128 and 136 rate bytes for SHAKE256. SHAKE256 and SHA3-256 therefore share a rate size but use different domain suffixes and output rules.

Exercises

Compute capacities for rate sizes (a) 168 bytes, (b) 136 bytes, (c) 72 bytes. How many 64-bit lanes are in the rate of SHA3-256?

4 Method 3: absorb one block

Input: rate bytes and current state
Do: XOR each little-endian message lane into the matching state lane
Output: updated state before permutation

Plain-language explanation

Absorption is not addition and not replacement. It is XOR. Capacity lanes receive no message bytes directly.

Mechanical rule

For every rate lane i : $S_i \leftarrow S_i \oplus M_i$. Then apply Keccak-f[1600].

Worked example 1 - zero state

State lane 0x00 XOR message lane 0xA5 becomes 0xA5.

Worked example 2 - nonzero state

0x3C XOR 0x0F = 0x33.

Worked example 3 - absorb cancels repeated bits

0xFF XOR 0xFF = 0x00.

Worked example 4 - little-endian lane

Bytes 01 02 03 04 05 06 07 08 load into the 64-bit lane 0x0807060504030201, then XOR with the state lane.

Exercises

Absorb into lanes: (a) 0x55 XOR 0xAA, (b) 0x12 XOR 0x34, (c) zero XOR 0x80, (d) state 0xFFFF0000FFFF0000 with message 0x00FF00FF00FF00FF.

5 Method 4: add domain suffix and final padding bit

Input: final partial rate block
Do: XOR domain byte at message end and 0x80 into last rate byte
Output: one full padded block

Plain-language explanation

Padding tells the sponge where the message ends and which function is being requested. Tyr-Crypto uses suffix 0x06 for SHA3 and 0x1F for SHAKE. The final rate byte also receives 0x80.

Mechanical rule

Copy the remaining message bytes into a zero block. XOR the domain byte into position $\text{rate} - 1$. If both target the same byte, XOR both values into that byte. XOR 0x80 into position $\text{rate} - 1$. If both target the same byte, XOR both values into that byte.

Worked example 1 - short SHA3 message

With toy rate 8 and message AA BB, block becomes AA BB 06 00 00 00 00 80.

Worked example 2 - short SHAKE message

Same message under SHAKE becomes AA BB 1F 00 00 00 00 80.

Worked example 3 - empty SHA3 message

Toy rate 8: 06 00 00 00 00 00 00 80.

Worked example 4 - only one free byte

If seven of eight bytes are message, the last byte receives 0x06 XOR 0x80 = 0x86.

Exercises

For toy rate 8, pad (a) empty SHAKE, (b) one-byte SHA3 message 00, (c) six-byte SHAKE message, (d) seven-byte SHAKE message.

6 Method 5: apply theta - mix information between columns

Input: 25 lanes
Do: XOR each column, derive five correction lanes, XOR correction into every lane of its column
Output: theta-updated state

Plain-language explanation

Theta first summarizes each column with XOR. Then each column receives information from its left neighbor and a one-bit rotation of its right neighbor. This makes a local change affect all five rows.

Mechanical rule

$$C[x] = A[x, 0] \oplus \dots \oplus A[x, 4].$$

$$D[x] = C[x - 1] \oplus \text{rotl}(C[x + 1], 1).$$

$$A[x, y] \leftarrow A[x, y] \oplus D[x]. \text{ Coordinates wrap modulo 5.}$$

Worked example 1 - 8-bit lanes

Column parities: (0x01, 0x00, 0x00, 0x00, 0x00).
 Corrections: (0x00, 0x01, 0x00, 0x00, 0x02).
 First output row: 0x01, 0x01, 0x00, 0x00, 0x02.

Worked example 2 - 8-bit lanes

Column parities: (0x1F, 0x00, 0x00, 0x00, 0x00).
 Corrections: (0x00, 0x1F, 0x00, 0x00, 0x3E).
 First output row: 0x01, 0x1F, 0x00, 0x00, 0x3E.

Worked example 3 - 8-bit lanes

Column parities: (0x14, 0x09, 0x0E, 0x03, 0x08).
 Corrections: (0x1A, 0x08, 0x0F, 0x1E, 0x2B).
 First output row: 0x1A, 0x09, 0x0D, 0x1D, 0x2F.

Worked example 4 - 8-bit lanes

Column parities: (0xFF, 0x00, 0xFF, 0x00, 0xFF).
 Corrections: (0xFF, 0x00, 0x00, 0x00, 0xFF).
 First output row: 0x00, 0x00, 0xFF, 0x00, 0x00.

Exercises

Using 4-bit lanes, compute theta for: (a) only $A[0, 0] = 1$, (b) first row all 1, (c) first column all 1, (d) explain why a correction is identical for all five lanes in one column.

7 Method 6: apply rho - rotate every lane by its assigned distance

Input: 25 lanes
Do: rotate each lane independently
Output: 25 rotated lanes

Plain-language explanation

Rho changes bit positions inside a lane. It does not change lane coordinates. Each coordinate has a fixed public rotation distance.

Worked example 1 - offset 0

Lane (0,0) has offset 0, so 0xA5 remains 0xA5.

Worked example 2 - offset 1

With 8-bit toy lanes, 0x81=10000001 rotated left 1 becomes 0x03.

Worked example 3 - real offset reduced for a toy lane

Official offset 62 on a 64-bit lane is used exactly as 62. For an 8-bit demonstration, $62 \bmod 8 = 6$; 0x12 rotated left 6 becomes 0x84.

Worked example 4 - offset 28

For an 8-bit toy lane, $28 \bmod 8 = 4$; 0xAB becomes 0xBA.

Exercises

Rotate 8-bit lanes: (a) 0x40 by 1, (b) 0xF0 by 4, (c) 0x01 by 7, (d) 0xAA by any odd distance and describe the pattern.

8 Method 7: apply pi - move lanes to new coordinates

Input: one lane at (x, y)
Do: send it to $(y, 2x + 3y \bmod 5)$
Output: same lane value at a new place

Plain-language explanation

Pi moves whole lanes. Rho rotates inside lanes; pi moves the resulting lanes. Tyr-Crypto combines both steps into direct assignments for speed.

Worked example 1

$(0, 0) \rightarrow (0, 0)$.

Worked example 2

$(1, 0) \rightarrow (0, 2)$ because $2 \cdot 1 + 0 = 2$.

Worked example 3

$(2, 3) \rightarrow (3, (4 + 9) \bmod 5) = (3, 3)$.

Worked example 4

$(4, 4) \rightarrow (4, (8 + 12) \bmod 5) = (4, 0)$.

Exercises

Find destinations for (a) $(0, 1)$, (b) $(3, 1)$, (c) $(1, 4)$, (d) show that each source has one destination.

9 Method 8: apply chi - nonlinear row mixing

Input: five lanes in one row
Do: for each lane, inspect the next two lanes
Output: five updated lanes

Mechanical rule

$A[x] \leftarrow B[x] \oplus ((\neg B[x+1]) \wedge B[x+2])$, with row indices modulo 5.

b_0	b_1	b_2	$b_0 \oplus ((\neg b_1) \wedge b_2)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Worked example 1 - one bit

$(b_0, b_1, b_2) = (0, 0, 1)$ gives $0 \oplus (1 \wedge 1) = \boxed{1}$.

Worked example 2 - another bit

$(1, 0, 1)$ gives $1 \oplus 1 = \boxed{0}$.

Worked example 3 - byte lanes

$B_0 = 0xAA$, $B_1 = 0xF0$, $B_2 = 0x0F$. Then $\neg B_1 = 0x0F$, AND $B_2 = 0x0F$, XOR B_0 gives $\boxed{0xA5}$.

Worked example 4 - complete 5-byte row

For row $00\ FF\ 0F\ F0\ AA$, applying the rule cyclically gives $\boxed{00\ 0F\ 05\ F0\ 55}$.

Exercises

Apply chi to bit triples (a) 001, (b) 101, (c) 110. Then compute the first output lane for row $12\ 34\ 56\ 78\ 9A$.

10 Method 9: apply iota - inject the round constant

Input:	state and round number
Do:	XOR one public constant into lane (0,0)
Output:	round-separated state

Plain-language explanation

iota changes only lane 0. The round constants prevent all rounds from being identical copies of one another.

Worked example 1

Lane 0 0x00 XOR round constant 0x01 becomes 0x01.

Worked example 2

0xA5 XOR 0x01=0xA4.

Worked example 3

0x0000000000000000 XOR real round constant 0x0000000000008082 becomes that constant.

Worked example 4

All lanes other than index 0 remain byte-for-byte unchanged during iota.

Exercises

Start with lane 0 equal to 0x00. Compute it after XOR with (a) 0x01, (b) 0x82, (c) 0x8A. Which array index changes?

11 Method 10: run one complete Keccak round

Input: 25-lane state and round constant
Do: θ then ρ then π then χ then ι
Output: next state

Mechanical rule

The order is fixed: $\theta \rightarrow \rho \rightarrow \pi \rightarrow \chi \rightarrow \iota$. SHA3 and SHAKE apply 24 such rounds per permutation call.

Worked example 1 - one toy round with 8-bit lanes and RC=01

Input first row: 0x00, 0x00, 0x00, 0x00, 0x00.

Output first row: 0x01, 0x00, 0x00, 0x00, 0x00.

This is a teaching-sized lane width; the real code uses identical Boolean structure on 64-bit lanes.

Worked example 2 - one toy round with 8-bit lanes and RC=01

Input first row: 0x01, 0x00, 0x00, 0x00, 0x00.

Output first row: 0x00, 0x10, 0x80, 0x01, 0x90.

This is a teaching-sized lane width; the real code uses identical Boolean structure on 64-bit lanes.

Worked example 3 - one toy round with 8-bit lanes and RC=01

Input first row: 0x01, 0x02, 0x03, 0x04, 0x05.

Output first row: 0x01, 0x51, 0xD0, 0x21, 0xF1.

This is a teaching-sized lane width; the real code uses identical Boolean structure on 64-bit lanes.

Worked example 4 - one toy round with 8-bit lanes and RC=01

Input first row: 0x00, 0x01, 0x02, 0x03, 0x04.

Output first row: 0x03, 0x61, 0x54, 0x93, 0x2C.

This is a teaching-sized lane width; the real code uses identical Boolean structure on 64-bit lanes.

Exercises

For a one-bit-per-lane toy, execute a full round on (a) all zeros with RC 1, (b) only lane 0 set, (c) first row set, (d) explain which steps are linear over XOR and which step is nonlinear.

12 Method 11: squeeze output bytes

Input:	permuted state and requested length
Do:	serialize rate lanes; permute again when rate is exhausted
Output:	fixed digest or arbitrary-length XOF output

Plain-language explanation

Squeezing reads only the rate part. SHA3 stops after a fixed digest length. SHAKE continues for as many output bytes as requested, applying another permutation each time one rate block has been consumed.

Worked example 1 - one lane

State lane 0x0807060504030201 squeezes bytes 01 02 03 04 05 06 07 08.

Worked example 2 - output shorter than rate

SHA3-256 requests 32 bytes while its rate is 136 bytes. One permutation output supplies the entire digest.

Worked example 3 - SHAKE crosses a rate boundary

SHAKE128 rate is 168 bytes. Requesting 200 bytes reads 168, permutes again, then reads 32 more.

Worked example 4 - zero-length request

A SHAKE API may validly request zero output bytes; no state bytes need be copied.

Exercises

How many squeeze blocks are needed for SHAKE256 outputs of (a) 1, (b) 136, (c) 137, and (d) 500 bytes?

13 Method 12: choose SHA3 or SHAKE and follow the whole sponge

Input:	message and function variant
Do:	block, pad, absorb, permute, squeeze
Output:	digest or extendable output

Worked example 1 - empty SHA3-256

Create one padded 136-byte block beginning 0x06 and ending with 0x80, absorb into zero state, permute once, then take 32 bytes.

Worked example 2 - 136-byte SHA3-256 message

The full 136-byte message is absorbed and permuted. A separate final padding block is then absorbed and permuted because the message exactly filled the rate.

Worked example 3 - short SHAKE128 output

Use rate 168 and suffix 0x1F; after final permutation, read the requested bytes. The output length is not encoded into the input by basic SHAKE.

Worked example 4 - long SHAKE256 output

Use rate 136 and suffix 0x1F. For 300 output bytes, read 136, permute, read 136, permute, read 28.

Exercises

For each case state rate, suffix, number of input permutation calls, and squeeze blocks: (a) empty SHA3-512, (b) 72-byte SHA3-512 message, (c) 10-byte SHAKE128 message and 200-byte output, (d) 300-byte SHAKE256 message and 32-byte output.

14 Reading the Tyr-Crypto implementation

Source item	Calculation meaning
Sha3State = array[25,uint64]	Method 1: flat 5 by 5 lane state.
sha3RateBytes	Method 2: fixed rate table for SHA3 variants.
absorbBlock	Method 3: XOR little-endian 64-bit message lanes into rate lanes.
absorbFinalBlockWithDomain	Method 4: domain suffix and final 0x80.
keccakF1600Ref column parities	Method 5 theta.
Direct b0..b24 assignments	Methods 6 and 7 rho plus pi.
chiRow template	Method 8 nonlinear row rule.
a0 xor roundConstant	Method 9 iota.
squeezeBytesState	Method 11 rate serialization and repeated permutations.

Do not skip this warning

Tyr-Crypto disables bounds and overflow checks inside a caller-validated hot region. This is an optimization assumption: correctness now depends on all rate sizes, buffer lengths, and indices being validated before entering that region.

15 Cumulative exercises

Cumulative A - toy sponge

Use an 8-byte rate and an 8-byte toy state. Absorb message AA BB, add SHA3 padding, XOR into zero state, then apply the toy permutation $P(x_0, \dots, x_7) = (x_1, \dots, x_7, x_0 \oplus 1)$. Squeeze four bytes.

Cumulative B - one toy Keccak round

Start with 25 zero 8-bit lanes except $A[0,0] = 1$. Compute theta column parities and corrections, then follow rho/pi, chi, and iota with RC 1. Use the tables in Methods 5–10.

Cumulative C - real SHAKE planning

SHAKE256 receives 300 bytes and must output 400 bytes. Determine input block structure, padding location in the final block, number of absorption permutations, and number of squeeze blocks.

Cumulative D - domain confusion

A developer changes SHA3 suffix 0x06 to SHAKE suffix 0x1F but still truncates to 32 bytes. Explain why the result is not SHA3-256 even though the output length matches.

16 Solutions

Methods 1–4: state, rate, absorption, and padding

Method 1. Use $i = x + 5y$: (a) 6; (b) 13; (c) 24; (d) $17 = 2 + 5 \cdot 3$, so $(x, y) = (2, 3)$.

Method 2. Capacity is $1600 - r$: rates 168, 136, 72 bytes give capacities 256, 512, 1024 bits. SHA3-256 has $136/8 = 17$ rate lanes.

Method 3. Absorption is XOR: (a) 0xFF; (b) 0x26; (c) 0x80; (d) 0xFF0000FFFF0000FF.

Method 4. With toy rate 8: (a) empty SHAKE becomes 1F 00 00 00 00 00 00 80; (b) one-byte SHA3 message 00 becomes 00 06 00 00 00 00 00 80; (c) six SHAKE message bytes are followed by 1F 80; (d) with seven bytes, suffix and final bit share the last byte, so it is XORed with $0x1F \oplus 0x80 = 0x9F$.

Methods 5–9: the five Keccak round steps

Method 5. Use 4-bit rotations. (a) $C = (1, 0, 0, 0, 0)$ and $D = (0, 1, 0, 0, 2)$. Every lane in column 1 gains 1 and every lane in column 4 gains 2; the original (0, 0) lane remains 1. (b) $C = (1, 1, 1, 1, 1)$ and every $D[x] = 3$; row 0 becomes five 2s and the other rows five 3s. (c) Again $C = (1, 0, 0, 0, 0)$ and $D = (0, 1, 0, 0, 2)$; column 0 stays all 1, column 1 becomes all 1, and column 4 becomes all 2. (d) $D[x]$ is computed once from column parities and then XORed into all five rows of column x .

Method 6. (a) 0x80; (b) 0x0F; (c) 0x80; (d) 0xAA = 10101010₂ becomes 0x55 after any odd rotation because the alternating pattern changes phase.

Method 7. (a) $(0, 1) \mapsto (1, 3)$; (b) $(3, 1) \mapsto (1, 4)$; (c) $(1, 4) \mapsto (4, 4)$. (d) The destination is fixed for every source; the inverse is $y = x'$ and $x = x' + 3y' \pmod{5}$, so two different sources cannot collide.

Method 8. Bit triples 001, 101, 110 produce 1, 0, 1. For 12 34 56 78 9A, the first lane is $0x12 \oplus ((\neg 0x34) \wedge 0x56) = 0x50$.

Method 9. Starting from zero, results are 0x01, 0x82, and 0x8A. Only flat array index 0, corresponding to (0, 0), changes.

Method 10: one-bit complete rounds

(a) All-zero input remains zero through theta, rho, pi, and chi; iota sets only (0, 0) to 1.

(b) With only lane 0 initially set, the five output rows are
0 1 1 0 1 0 0 0 1 1 1 1 0 0 0 0 0 1 1 0 1 0 0 0 1.

(c) With the first row set, output row 0 is 0 0 0 1 0; each of rows 1–4 is 1 0 0 1 0.

(d) Theta, rho, and pi are linear over XOR. Chi is nonlinear because it contains AND. Iota is addition of a fixed XOR constant, so it is affine rather than strictly linear.

Methods 11–12: squeezing and complete function planning

Method 11. SHAKE256 has a 136-byte rate. Requested lengths 1, 136, 137, 500 therefore need 1, 1, 2, 4 squeeze blocks.

Method 12. (a) Empty SHA3-512: rate 72, suffix 0x06, one padded input permutation, one squeeze block. (b) A 72-byte SHA3-512 message: one full-block permutation plus one empty padded-block permutation, then one squeeze block. (c) Ten-byte SHAKE128: rate 168, suffix 0x1F, one input permutation, and two output blocks for 200 bytes. (d) 300-byte SHAKE256: rate 136, suffix 0x1F, two full input blocks plus one 28-byte final block, hence three input permutations and one squeeze block for 32 output bytes.

Cumulative solutions A–B

A. Padded block: AA BB 06 00 00 00 00 80. XORing into zero gives the same state. The toy permutation produces BB 06 00 00 00 00 80 AB; squeeze BB 06 00 00.

B. For 8-bit teaching lanes, theta has $C = (0x01, 0, 0, 0, 0)$ and $D = (0, 0x01, 0, 0, 0x02)$. After rho and pi, the rows are

01 10 00 00 80 00 20 00 20 00 02 00 00 02 00 10 00 04 00 00 00 00 01 00 04.

After chi and iota with RC 1, the final rows are

00 10 80 01 90 00 00 00 20 20 02 02 00 00 00 14 00 04 10 00 01 00 05 00 04.

Cumulative solutions C–D

C. SHAKE256 rate is 136 bytes. The 300-byte input contains two full blocks (272 bytes) and a final 28-byte block holding suffix and final padding: three absorption permutations. Four hundred output bytes require three squeeze blocks, $136 + 136 + 128$.

D. The suffix is part of the absorbed input and selects the function domain. Replacing 0x06 with 0x1F changes the internal state before permutation. A 32-byte truncation of SHAKE256 therefore remains truncated SHAKE256, not SHA3-256.

17 References and code map

Source	Link
Tyr SHA3/SHAKE source	https://github.com/siriuslee69/Tyr-Crypto/blob/fb9cf27926140e95c02362d4fe1c349ff65d3dd0/src/protocols/custom_crypto/symmetric/sha3/sha3.nim
Tyr SHA3 SIMD source	https://github.com/siriuslee69/Tyr-Crypto/blob/fb9cf27926140e95c02362d4fe1c349ff65d3dd0/src/protocols/custom_crypto/symmetric/sha3/sha3_simd.nim
FIPS 202 Keccak specifications summary	https://csrc.nist.gov/pubs/fips/202/final https://keccak.team/keccak_specs_summary.html

Do not skip this warning

This handbook explains calculations and repository structure. It is not a security certification. Do not deploy custom cryptographic code solely because worked examples match.